UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

Departamento de Ciencias de la Computación



Ingeniería de Software

INGENIERÍA DE SOFTWARE

RESUMEN DESCRIPTIVO DE LA UNIDAD 1

Introducción y estudio de vulnerabilidades

Asignatura	Desarrollo de Software Seguro
Unidad	Unidad 1: Introducción y estudio de vulnerabilidades
Actividad	Resumen descriptivo y documentado
Modalidad	Trabajo individual
Estudiante	Erick Alejandro Tufiño Ortiz
Docente	Geovanny Cudco

2026

Índice

Introducción general **1**

1. Unidad 1: Introducción y estudio de vulnerabilidades **1**

1.1. Introducción 1

1.1.1. Qué es la seguridad del software 1

1.1.2. La importancia de la seguridad en el software 2

1.1.3. Las vulnerabilidades y sus costes 2

1.1.4. Gestión de riesgos de seguridad del software 3

1.1.5. Propiedades del software seguro y resiliente 4

1.1.6. Conceptos de seguridad 4

1.1.7. Servidores web HTTP, bases de datos 5

1.2. Amenazas a la seguridad del software 5

1.2.1. Amenazas a nivel de hardware 5

1.2.2. Amenazas a nivel de código 6

1.2.3. Lenguajes de script incrustados 7

1.2.4. Amenazas a nivel de diseño 7

1.2.5. Amenazas a nivel de arquitectura 8

1.3. Plan estratégico 8

1.3.1. Seguridad y resiliencia en el SDLC 9

1.3.2. Directrices de seguridad adicionales 9

1.3.3. Puntos de ataque y seguridad perimetral 10

Síntesis comparativa de amenazas 10

Síntesis de seguridad en el SDLC 11

Conclusiones **12**

Referencias **13**

Introducción general

El software participa en servicios de comunicación, educación, comercio y gestión de información. Por ello, un producto que satisface requisitos funcionales no es necesariamente seguro: también debe proteger los activos que procesa, limitar acciones no autorizadas y conservar funciones esenciales ante fallas o ataques. La seguridad requiere decisiones coordinadas sobre requisitos, diseño, implementación, operación y mantenimiento, en lugar de una inspección aislada al final del proyecto [1], [2].

Este resumen organiza los fundamentos de seguridad del software, las amenazas que pueden originarse en distintas capas y las acciones para integrarla al ciclo de vida del desarrollo (SDLC, por sus siglas en inglés). Se relacionan activos, amenazas, debilidades, escenarios de riesgo y controles mediante ejemplos de aplicaciones web. La exposición es documental y no reporta experimentos ni resultados de una implementación.

1 Unidad 1: Introducción y estudio de vulnerabilidades

1.1 Introducción

La seguridad es una propiedad transversal del software y un proceso de gestión. Su análisis parte de aquello que tiene valor, considera qué puede afectarlo y determina medidas proporcionales al riesgo. Esta perspectiva conecta la protección del producto con la continuidad del servicio y la capacidad de recuperarse, orientando controles verificables durante todo el SDLC [1], [3].

1.1.1 Qué es la seguridad del software

La seguridad del software comprende las propiedades y prácticas necesarias para que un programa o sistema funcione de manera confiable frente a errores, uso indebido, ataques y condiciones adversas. La corrección funcional responde si el sistema realiza lo especificado; la seguridad también pregunta si lo hace solo bajo condiciones y permisos previstos. Una aplicación puede autenticar correctamente a sus usuarios y, aun así, exponer el perfil de otra persona si no comprueba la autorización para cada objeto solicitado.

La Figura 1 resume la integración de seguridad a lo largo del desarrollo y la operación. Un programa es un conjunto de instrucciones; una aplicación organiza software para tareas de usuario y un sistema informático incluye además infraestructura y otros componentes. La disciplina abarca requisitos de seguridad, arquitectura y diseño, codificación, pruebas, despliegue y mantenimiento. La seguridad por diseño implica que las propiedades de protección sean criterios explícitos desde la concepción del producto. McGraw sostiene la necesidad de integrar prácticas de seguridad al proceso de construcción y complementarlas con análisis y pruebas [4]. La ciberseguridad protege el ecosistema digital más amplio; la seguridad de aplicaciones se concentra en aplicaciones durante su ciclo de vida; y la seguridad de la información protege confidencialidad, integridad y disponibilidad de información en cualquier soporte. Se superponen, pero no son equivalentes: comparten gestión de riesgos, controles, monitoreo, respuesta y cumplimiento.

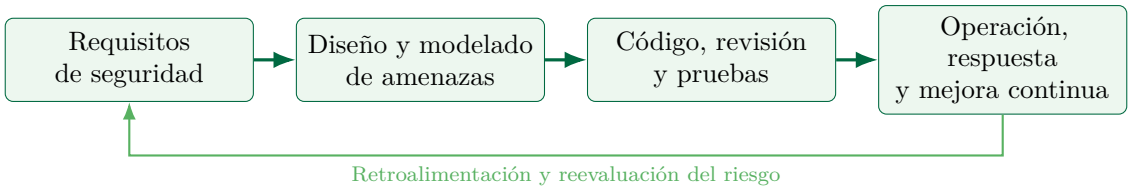


Figura 1: La seguridad como actividad continua del desarrollo y la operación. Elaboración propia a partir del SSDF [1].

1.1.2 La importancia de la seguridad en el software

Una debilidad puede comprometer datos personales, impedir el acceso a servicios, alterar operaciones o disminuir la confianza de quienes dependen del sistema. La importancia de la seguridad comprende proteger usuarios y organizaciones, preservar continuidad operativa y atender obligaciones legales o contractuales aplicables. La protección debe corresponder al valor de los activos y a escenarios plausibles, no a una acumulación indiscriminada de mecanismos.

Un programa equilibrado procura prevenir vulnerabilidades y reducir superficie expuesta; detectar debilidades o comportamientos anómalos mediante pruebas y monitoreo; responder para contener un incidente y limitar sus efectos; y recuperarse restaurando funciones y datos confiables. Los objetivos se complementan: prevenir no elimina todo riesgo y detectar sin responder no limita el daño. El enfoque reactivo trata problemas cuando el producto ya está desplegado, manteniendo exposición durante la investigación y pudiendo generar retrabajo; el preventivo identifica requisitos y controles temprano y los verifica en varias fases. Las prácticas integradas al SDLC ayudan a gestionar debilidades antes de producción [1], [3].

1.1.3 Las vulnerabilidades y sus costes

Una vulnerabilidad es una debilidad que puede ser aprovechada por una amenaza para producir un efecto adverso. Una debilidad puede estar en el código, diseño o configuración; no todo error es explotable, y la existencia de vulnerabilidad no significa que ya haya ocurrido una explotación. La explotación es el uso efectivo de una debilidad; un incidente es un evento que compromete o amenaza la seguridad; la consecuencia describe el daño resultante.

Las fuentes potenciales abarcan requisitos incompletos, arquitectura deficiente, errores de implementación, dependencias, configuraciones, infraestructura y mantenimiento. Los costos posibles comprenden investigación y remediación técnica; interrupción operativa y pérdida de productividad; gastos de respuesta y recuperación; consecuencias legales o regulatorias; y deterioro reputacional. Puede ser necesario restaurar información y validar su integridad. La magnitud depende del activo y del contexto, por lo que no corresponde atribuir cifras universales sin evidencia.

Las taxonomías ayudan a describir y priorizar, pero no son intercambiables. CVE (Common Vulnerabilities and Exposures) identifica vulnerabilidades divulgadas; CWE (Common Weakness Enumeration) clasifica debilidades; CVSS (Common Vulnerability Scoring System) expresa severidad técnica; NVD ofrece información estructurada de vulnerabilidades y productos; CPE (Common Platform Enumeration) identifica productos de forma normalizada; y CAPEC (Common Attack Pattern Enumeration and Classification) organiza patrones de ataque [5], [6]. La lista KEV

(Known Exploited Vulnerabilities) de CISA señala vulnerabilidades con evidencia de explotación activa, mientras EPSS (Exploit Prediction Scoring System) estima la probabilidad de explotación en un periodo próximo [7], [8]. La severidad técnica no equivale al riesgo organizacional: se consideran exposición, activos, controles, impacto y contexto. Un flujo útil es descubrir, clasificar, evaluar, priorizar, remediar y verificar.

1.1.4 Gestión de riesgos de seguridad del software

Un activo posee valor para una organización. Una amenaza es una entidad, evento o condición capaz de causar daño; una vulnerabilidad es una debilidad que facilita ese daño; la exposición describe la situación que permite que un activo quede alcanzable. El impacto son las consecuencias; la probabilidad estima la posibilidad del escenario. El riesgo combina probabilidad e impacto y un control reduce probabilidad, exposición o consecuencias.

Considérese una base de datos de usuarios: es el activo; una consulta SQL construida por concatenación insegura es la vulnerabilidad; un actor que envía datos manipulados constituye la amenaza; un formulario público aumenta exposición; y la lectura o modificación no autorizada es el escenario con impacto. Consultas parametrizadas, validación y autorización apropiada son controles. Esta relación —activo, amenaza, vulnerabilidad, escenario, riesgo y control— es útil sin implicar una sola cadena causal para todo riesgo.

La Figura 2 representa las relaciones principales para analizar un escenario, sin reducir todos los casos a una cadena causal única. El proceso identifica activos, amenazas y debilidades; analiza escenarios; estima probabilidad e impacto; evalúa y prioriza; selecciona tratamiento; implanta controles; y monitorea y reevalúa. El riesgo puede mitigarse, evitarse abandonando la actividad, transferirse o compartirse mediante acuerdos, o aceptarse con decisión informada. El riesgo inherente se valora antes de controles; el residual permanece después y se compara con criterios de aceptación.

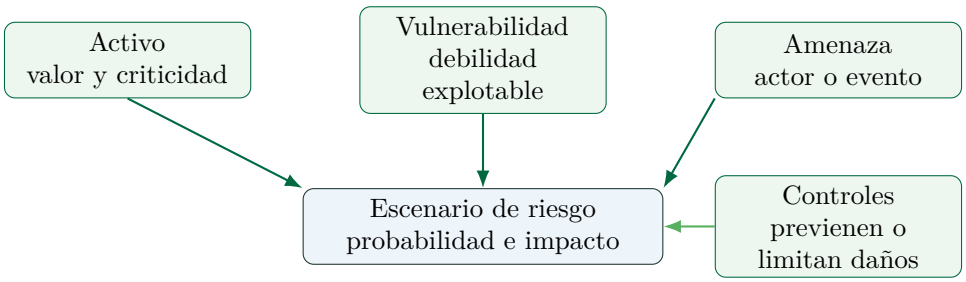


Figura 2: Factores que intervienen en un escenario de riesgo y papel de los controles.

Como aproximación cualitativa o semicuantitativa, con escalas de cinco niveles, puede expresarse la prioridad relativa de manera simplificada como:

$$\text{Riesgo} = \text{Probabilidad} \times \text{Impacto}$$

No es una fórmula universal ni produce probabilidades reales; requiere criterios consistentes y juicio contextual. NIST SP 800-30 describe una evaluación sistemática que informa decisiones y comunicación de riesgos [3].

1.1.5 Propiedades del software seguro y resiliente

La tríada confidencialidad, integridad y disponibilidad (CIA) orienta objetivos esenciales. Confidencialidad limita la información a sujetos autorizados; autorización y cifrado contribuyen a ella. Integridad protege datos y operaciones frente a modificaciones indebidas mediante validación y controles de acceso. Disponibilidad exige acceso oportuno para usuarios autorizados; redundancia y recuperación contribuyen a sostenerla.

La autenticidad permite verificar identidades y orígenes; la autenticación y la validación de certificados se relacionan con ella. La trazabilidad permite reconstruir actividades mediante registros protegidos y revisables. La resiliencia es la capacidad de anticipar, resistir, absorber, recuperarse y adaptarse frente a condiciones adversas. Se apoya en monitoreo, respaldos, continuidad y ejercicios de recuperación. La Figura 4 muestra cómo las capas preventivas, detectivas y de recuperación se complementan. No equivale a impedir todo incidente: busca mantener o restablecer funciones críticas y limitar consecuencias [3].



Figura 3: Propiedades de confidencialidad, integridad y disponibilidad explicadas mediante ejemplos.

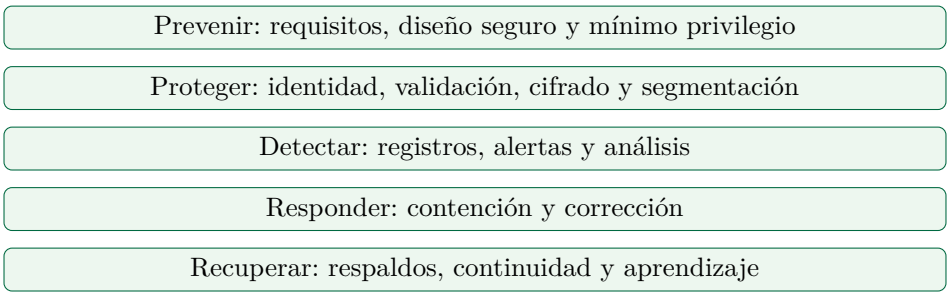


Figura 4: Capas complementarias de defensa y resiliencia. Elaboración propia.

1.1.6 Conceptos de seguridad

Autenticación verifica quién es un usuario, sistema o servicio; autorización decide qué recursos u operaciones puede utilizar. Iniciar sesión no debe otorgar automáticamente permisos administrativos. Un ataque intenta vulnerar propiedades; su vector es la vía empleada y la superficie de ataque reúne componentes accesibles que podrían aprovecharse. Un punto de entrada puede ser una API, formulario, puerto o integración externa.

Los controles pueden ser preventivos, detectivos o correctivos. Un evento de seguridad es una ocurrencia relevante; un incidente supone afectación o amenaza que requiere respuesta. Exposición es la condición de accesibilidad o susceptibilidad ante una amenaza y no equivale por sí sola a vulnerabilidad.

El mínimo privilegio limita permisos a lo necesario; la defensa en profundidad dispone controles complementarios; la separación de responsabilidades evita concentrar capacidades incompatibles;

y la denegación por defecto exige conceder accesos explícitamente. Zero Trust elimina la confianza implícita: “Nunca confiar implícitamente; verificar cada acceso”. Identidad, contexto y permisos se evalúan explícitamente, sin presumir confianza por pertenecer a una red interna [11]. La seguridad por diseño integra estas decisiones desde etapas tempranas.

1.1.7 Servidores web HTTP, bases de datos

En el modelo cliente-servidor, un cliente envía una petición HTTP y el servidor responde con estado, encabezados y, en su caso, contenido. Métodos comunes son GET para recuperar, POST para enviar o crear, PUT/PATCH para modificar y DELETE para eliminar. Los códigos de estado comunican éxito, redirección, error del cliente o servidor. HTTPS transporta HTTP protegido con TLS, aportando confidencialidad e integridad del canal y autenticación del servidor cuando se valida correctamente; no corrige fallas de autorización o lógica.

El servidor web recibe y enruta solicitudes a la aplicación, que aplica reglas de negocio y consulta un sistema gestor de bases de datos (SGBD). Los riesgos incluyen servicios innecesariamente expuestos, configuración débil, errores de autorización, consultas inseguras, credenciales reveladas, manejo deficiente de sesiones y falta de monitoreo. Controles: HTTPS, validación en servidor, consultas parametrizadas, privilegios mínimos, almacenamiento seguro de secretos, configuración endurecida, respaldos, registros y segmentación. Un proxy inverso o WAF complementa, pero no sustituye código correcto ni autorización interna [12], [13].

1.2 Amenazas a la seguridad del software

Las amenazas pueden ser externas o internas, deliberadas o accidentales; errores operativos y fallas técnicas también originan escenarios adversos. Clasificarlas por nivel —hardware, código, scripts, diseño y arquitectura— ayuda a relacionar el origen con consecuencias y controles. La clasificación no es excluyente: una falla arquitectónica puede amplificar una vulnerabilidad de código.

1.2.1 Amenazas a nivel de hardware

El hardware, firmware, interfaces y procedencia forman parte de la superficie de ataque. Acceso físico no autorizado, robo o manipulación, firmware comprometido, periféricos maliciosos, interfaces de depuración expuestas y fallas físicas pueden afectar confidencialidad, integridad, disponibilidad y confianza en el arranque. Un firmware alterado antes del inicio podría persistir fuera del alcance de controles ordinarios del sistema operativo.

BIOS/UEFI participa en la inicialización; Secure Boot verifica componentes de arranque según una cadena de confianza; TPM puede proteger claves y mediciones; SSD/HDD almacenan datos; USB y DMA son interfaces que requieren control. Ningún mecanismo elimina todas las amenazas físicas. Se complementan con seguridad física, inventario, control de dispositivos, actualización de firmware, protección de claves, arranque verificado, trazabilidad de proveedores, monitoreo de integridad y respaldos. La cadena de suministro debe considerarse antes de que los equipos lleguen a operación [2], [3].



Figura 5: Superficies de ataque relacionadas con firmware, memoria, almacenamiento y dispositivos.

1.2.2 Amenazas a nivel de código

Entradas no confiables sin validación contextual pueden producir inyecciones. En SQL injection, concatenar texto recibido con una consulta puede cambiar su estructura; consultas parametrizadas separan datos e instrucciones. La inyección de comandos ocurre cuando datos manipulables alcanzan operaciones del sistema. Otros problemas incluyen autenticación débil, autorización insuficiente, errores que revelan información, secretos en repositorios, fallos criptográficos, defectos de memoria y dependencias vulnerables. CWE clasifica debilidades y OWASP organiza riesgos relevantes para aplicaciones web [5], [12].

La mitigación combina codificación segura, autorización en servidor, manejo controlado de errores, gestión de secretos fuera del código, mínimo privilegio, actualización de componentes y revisión. SAST analiza código o artefactos sin ejecutar la aplicación; DAST examina una aplicación en ejecución; SCA identifica componentes y dependencias de terceros y sus riesgos conocidos. Ninguna técnica prueba por sí sola ausencia de vulnerabilidades; se complementan con pruebas funcionales, revisión humana y verificación de correcciones.

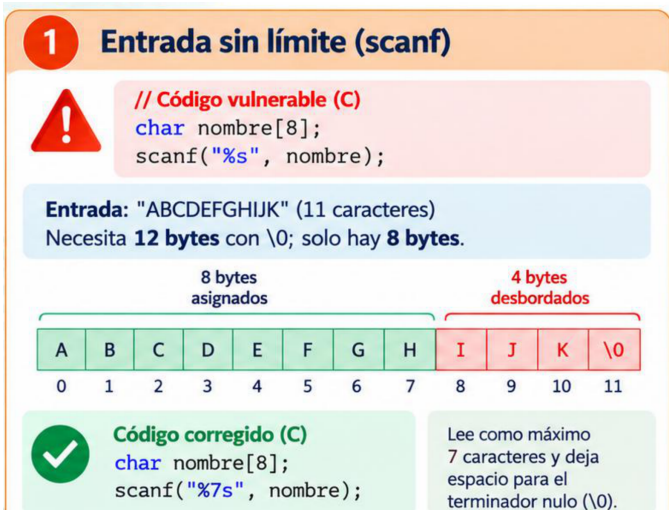


Figura 6: Ejemplo didáctico de un desbordamiento de búfer por ausencia de límites en la entrada.

1.2.3 Lenguajes de script incrustados

Los scripts se ejecutan en navegador o servidor, y las plantillas procesan expresiones para producir contenido dinámico. En XSS, contenido no confiable se interpreta como código en el contexto del sitio. DOM XSS puede surgir cuando el código cliente inserta datos no confiables en operaciones peligrosas del DOM. Usar innerHTML puede interpretar una cadena como marcado; textContent la incorpora como texto en ese contexto.

La SSTI ocurre cuando una entrada no confiable se evalúa como expresión por un motor de plantillas. También es riesgosa la ejecución dinámica como eval() y la carga de scripts externos sin control de integridad. Medidas útiles son separar datos de plantillas, codificar salida según contexto, sanitizar solo cuando se admite HTML, evitar evaluación dinámica y limitar scripts con CSP. La integridad de subrecursos ayuda a verificar recursos externos. Validar entrada no sustituye codificar salida, y CSP no corrige automáticamente una aplicación vulnerable [15], [16].

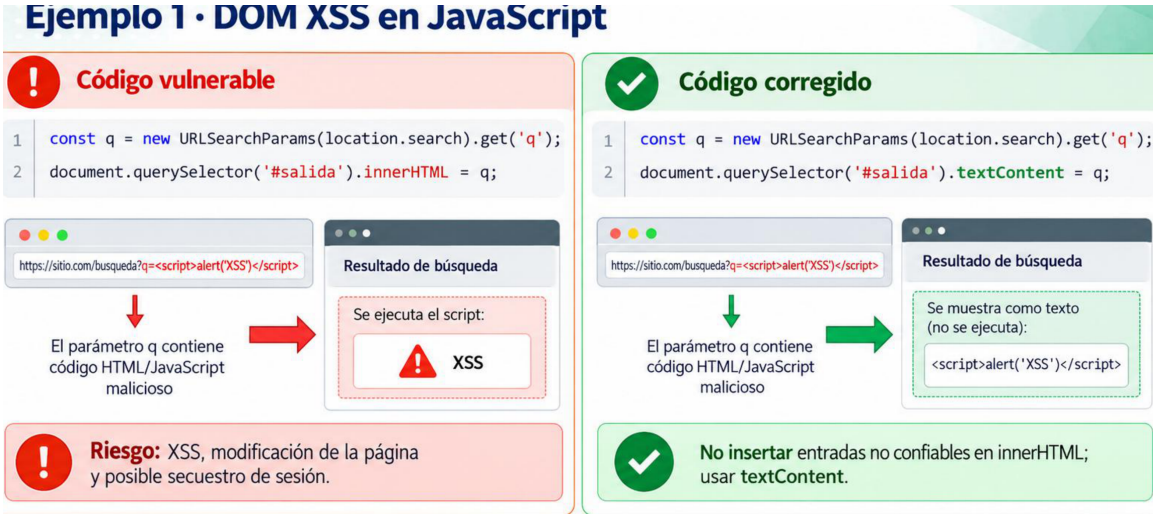


Figura 7: Comparación entre inserción insegura mediante `innerHTML` y tratamiento como texto mediante `textContent`.

1.2.4 Amenazas a nivel de diseño

Los defectos de diseño se originan en requisitos y flujos que permiten comportamientos inseguros aun cuando el código implemente lo previsto. Ejemplos: omitir autorización por objeto, confiar solo en validaciones del cliente, carecer de límites de confianza o permitir permisos excesivos. En una tienda, si el proceso no define límites para descuentos, puede permitirse su aplicación ilimitada sin error de sintaxis: la debilidad está en la regla de negocio.

El modelado de amenazas comprende entender activos y procesos; localizar entradas y límites de confianza; identificar amenazas y abuso; estimar consecuencias; definir controles; y verificar decisiones. STRIDE organiza suplantación, manipulación, repudio, divulgación de información, denegación de servicio y elevación de privilegios [17]. Se relaciona con Secure by Design, mínimo privilegio, denegación por defecto, autorización completa y separación de responsabilidades. Shostack aborda el modelado como práctica para descubrir amenazas y orientar decisiones de diseño [18].

1.2.5 Amenazas a nivel de arquitectura

La arquitectura determina distribución, interacción y límites de confianza entre componentes; el diseño concreta comportamientos y controles. Amenazas arquitectónicas incluyen servicios innecesariamente expuestos, comunicación sin protección, segmentación deficiente, confianza implícita entre servicios, puntos únicos de falla, aislamiento insuficiente, dependencias externas excesivas y gestión insegura de identidades. Una falla puede propagarse y comprometer disponibilidad o datos de otros servicios.

La Figura 9 presenta una arquitectura web conceptual y controles de cada capa. Considérese una arquitectura de microservicios con cliente, API Gateway, autenticación, usuarios, operaciones y base de datos. El Gateway enruta y aplica controles comunes, pero no sustituye la autorización particular del servicio sobre cada recurso. Si un servicio acepta sin verificar una petición de otro componente interno, un atacante que comprometa un nodo podría desplazarse lateralmente. Identidades de servicio, comunicación protegida —autenticación mutua cuando corresponda—, autorización por recurso, segmentación, aislamiento, gestión de secretos, monitoreo distribuido, redundancia y recuperación reducen riesgos. Zero Trust exige verificar comunicaciones explícitamente, no confiar por ubicación en la red [2], [11].

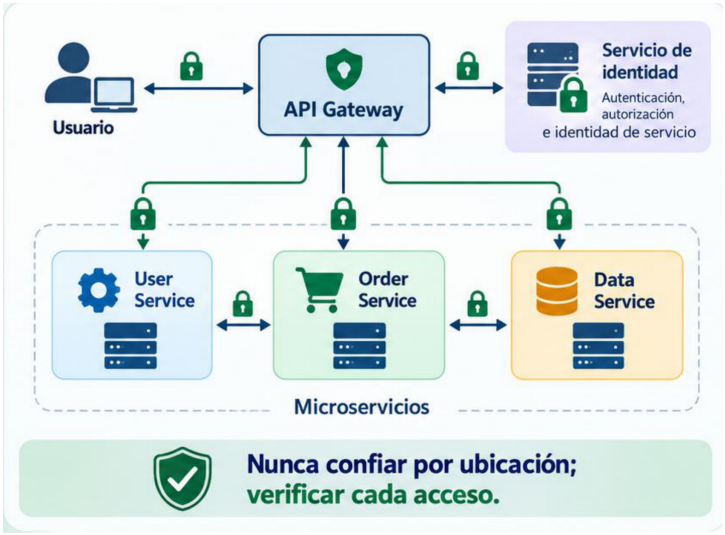


Figura 8: Aplicación conceptual de Zero Trust a una arquitectura de microservicios.

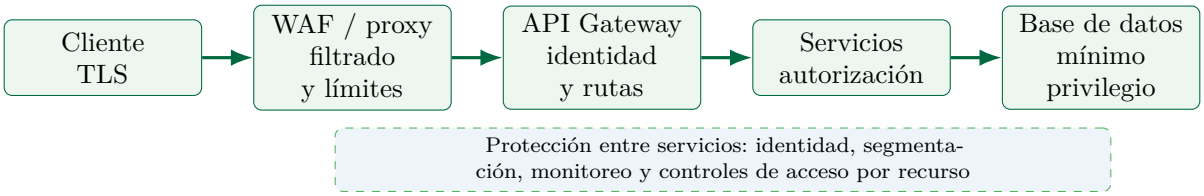


Figura 9: Arquitectura web conceptual con controles distribuidos por capa.

1.3 Plan estratégico

La planificación debe partir de objetivos y riesgos del sistema, definir responsabilidades y asignar verificaciones durante su construcción y operación. No consiste en añadir una barrera final, sino en coordinar prevención, detección, respuesta y recuperación para activos relevantes.

El SDLC ofrece puntos de decisión donde revisar requisitos, controles y evidencia [1], [2].

1.3.1 Seguridad y resiliencia en el SDLC

En requisitos se identifican activos, restricciones, reglas de acceso y criterios de confidencialidad y disponibilidad. Historias de usuario pueden incorporar límites de intentos, doble factor o expiración de sesión según el riesgo. Los requisitos de seguridad pueden ser funcionales o no funcionales según el comportamiento especificado; no todos se clasifican igual.

En diseño se modelan amenazas, límites de confianza, permisos, autenticación, arquitectura y recuperación. En implementación se aplican codificación segura, validación, consultas parametrizadas, gestión de secretos, revisión y análisis estático. En pruebas se verifican requisitos, casos negativos, autorización, SAST, DAST, SCA y escenarios de abuso. En despliegue se endurece configuración, se protege CI/CD y secretos, se limita exposición y se configura TLS. En operación y mantenimiento se monitorea, actualizan dependencias, gestionan vulnerabilidades, responden incidentes y prueban respaldos y recuperación. La Figura 10 sintetiza esta secuencia y una práctica característica por etapa.

Shift Left promueve incorporar verificaciones temprano; Secure by Design prevé seguridad en requisitos y estructura; el SSDF del NIST reúne prácticas organizacionales y técnicas para reducir vulnerabilidades y mitigar el impacto de las no detectadas [1]. Son conceptos relacionados, pero distintos: momento de incorporación, intención de diseño y marco de prácticas. Las medidas preventivas se integran en requisitos, diseño y codificación; las detectivas en revisión, pruebas y operación; las correctivas en respuesta y recuperación.

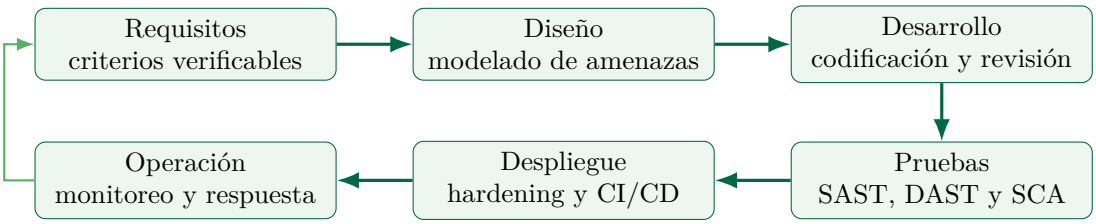


Figura 10: Seis fases del SDLC y una práctica característica de seguridad.

1.3.2 Directrices de seguridad adicionales

El mínimo privilegio limita daño si una cuenta o componente es comprometido. La defensa en profundidad combina controles de identidad, aplicación, red y datos para no depender de una barrera. Configuración segura y parches reducen servicios y componentes vulnerables; gestión de dependencias identifica riesgos heredados; los secretos deben mantenerse fuera del código fuente. Autenticación multifactor añade una barrera para accesos de riesgo elevado.

Segmentación limita movimiento entre componentes; monitoreo y auditoría facilitan detección e investigación. Respaldos comprobados y continuidad permiten restaurar funciones, con protección contra alteración y pruebas periódicas. Zero Trust aplica verificación explícita en lugar de confiar por ubicación de red. Estas directrices resuelven problemas diferentes y se eligen según el riesgo.

Un SGSI organiza políticas, procesos, responsabilidades y controles para gestionar riesgos de información y mejorarlos continuamente. ISO/IEC 27001:2022 establece requisitos de un sistema de gestión; ISO/IEC 27002:2022 ofrece controles de referencia; ISO/IEC 27005:2022

aborda gestión de riesgos de seguridad de la información. NIST CSF 2.0 organiza resultados para gestionar riesgos de ciberseguridad [19]; NIST SSDF se enfoca en prácticas de desarrollo seguro [1], [20, 21, 22]. No cumplen la misma función: se complementan entre gobierno, controles, riesgos y construcción de software. Planificar, Hacer, Verificar y Actuar (PHVA) representa mejora continua: definir medidas, implementarlas, revisar eficacia y ajustarlas.

1.3.3 Puntos de ataque y seguridad perimetral

La superficie de ataque incluye componentes, datos e interfaces accesibles; los puntos de entrada son lugares concretos como endpoints HTTP, APIs REST o GraphQL, formularios, autenticación, tokens y sesiones, cargas de archivos, puertos, consolas administrativas, integraciones y conexiones a bases de datos. Inventariarlos ayuda a determinar dónde aplicar controles.

Los cortafuegos restringen tráfico de red; un WAF inspecciona solicitudes web; un proxy inverso centraliza terminación TLS y enrutamiento; autenticación y autorización controlan identidades y acciones; rate limiting reduce abuso de recursos; segmentación y restricción de puertos limitan exposición; validación y monitoreo protegen y observan la aplicación. El perímetro no impide abuso de cuentas válidas, fallas de autorización o vulnerabilidades entre servicios. Se requieren seguridad por capas y verificación explícita dentro de la red [11, 12, 13].

Síntesis comparativa de amenazas

La Tabla 1 compara los cinco niveles requeridos. Ejemplo y mitigación se emparejan según el mecanismo: controles de firmware no resuelven autorización y consultas parametrizadas no corrigen segmentación. En un escenario pueden coexistir debilidades de varios niveles. La comparación distingue la amenaza o debilidad del daño que puede ocasionar; la consecuencia depende de activos, exposición y controles. La mitigación debe responder al origen de la debilidad y no limitarse a añadir una defensa externa.

Tabla 1: Comparación de amenazas a la seguridad del software según su nivel de origen. Elaboración propia.

Nivel	Amenaza o ejemplo	Consecuencia posible	Medida de mitigación
Hardware	Firmware de arranque manipulado durante la entrega.	Persistencia del compromiso y pérdida de confianza en el inicio.	Verificación del firmware y del arranque; evaluación de proveedores.
Código	Consulta SQL construida mediante concatenación de entradas.	Lectura o modificación no autorizada de registros.	Consultas parametrizadas y autorización en el servidor.
Scripts incrustados	Datos no confiables interpretados como script en el navegador.	Acciones en el contexto del sitio y exposición de datos.	Codificación contextual, DOM seguro y CSP complementaria.
Diseño	Regla que omite autorización por objeto.	Acceso a información perteneciente a otra cuenta.	Especificar autorización por recurso y verificar cada operación.
Arquitectura	Servicios internos aceptan peticiones sin verificar su identidad.	Movimiento lateral y acceso indebido entre componentes.	Identidades de servicio, segmentación, autorización y monitoreo.

Síntesis de seguridad en el SDLC

La Tabla 2 relaciona seis fases con prácticas concretas y los problemas que ayudan a tratar. Su valor aumenta cuando se convierten en criterios verificables y cuando los hallazgos alimentan correcciones y mejoras [1], [3]. Las fases se retroalimentan: pruebas comprueban decisiones de diseño y requisitos; incidentes operativos indican ajustes en código y arquitectura; actualizaciones requieren nueva verificación. Así se articulan prevención, detección, respuesta y recuperación sin suponer que una herramienta aislada garantiza la seguridad.

Tabla 2: Integración de seguridad y resiliencia durante el ciclo de vida del software.

Fase del SDLC	Práctica de seguridad y resiliencia	Propósito o riesgo mitigado
Requisitos	Identificar activos, permisos y criterios de aceptación de seguridad.	Prevenir omisiones y hacer verificables las necesidades desde el inicio.
Diseño	Modelado de amenazas, límites de confianza y estrategia de recuperación.	Evitar flujos inseguros y puntos únicos de falla; planificar continuidad.
Desarrollo	Codificación segura, consultas parametrizadas, revisión de código y secretos gestionados.	Reducir inyecciones, credenciales expuestas y defectos de implementación.
Pruebas	SAST, DAST, SCA y pruebas negativas de autorización.	Detectar defectos y dependencias vulnerables antes del despliegue.
Despliegue	Hardening, CI/CD protegido, TLS y restricción de servicios.	Disminuir la superficie expuesta y proteger la configuración operativa.
Operación y mantenimiento	Monitoreo, parches, respuesta a incidentes, respaldos y pruebas de recuperación.	Detectar, contener, restaurar el servicio y aprender de los incidentes.

Conclusiones

La seguridad del software exige proteger activos y propiedades como confidencialidad, integridad, disponibilidad, autenticidad y trazabilidad, sin confundir funcionamiento correcto con comportamiento seguro. Relacionar amenazas, vulnerabilidades, exposición, impacto y controles permite construir escenarios de riesgo y priorizar tratamientos. Las taxonomías apoyan la comunicación, pero la severidad técnica debe interpretarse en contexto organizacional.

Las amenazas pueden originarse en hardware, código, scripts, diseño o arquitectura, y sus consecuencias dependen de la interacción entre capas. Los controles deben corresponder al mecanismo: proteger firmware, validar entradas, especificar autorización por objeto y verificar comunicaciones entre servicios. Ninguna defensa única —incluido un perímetro, WAF o mecanismo de arranque— elimina todas las amenazas.

Integrar seguridad y resiliencia en requisitos, diseño, implementación, pruebas, despliegue y operación permite gestionar debilidades continuamente. La prevención se complementa con detección, respuesta y recuperación: aun si un incidente no puede evitarse, la organización puede limitar su propagación, restablecer funciones y aprender. La gestión del riesgo residual y la mejora continua mantienen las decisiones alineadas con cambios en el sistema y su entorno.

Referencias

[1] M. Souppaya, K. Scarfone y D. Dodson, *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*, NIST SP 800-218, 2022, doi: 10.6028/NIST.SP.800-218.

[2] R. Ross, M. Winstead y M. McEvilley, *Engineering Trustworthy Secure Systems*, NIST SP 800-160, vol. 1, rev. 1, 2022, doi: 10.6028/NIST.SP.800-160v1r1.

[3] Joint Task Force Transformation Initiative, *Guide for Conducting Risk Assessments*, NIST SP 800-30, rev. 1, 2012, doi: 10.6028/NIST.SP.800-30r1.

[4] G. McGraw, *Software Security: Building Security In*. Addison-Wesley Professional, 2006, ISBN: 9780321356703.

[5] MITRE, "2025 CWE Top 25 Most Dangerous Software Weaknesses," 2025. [En línea]. Disponible: `https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25`.

[6] National Institute of Standards and Technology, "National Vulnerability Database." [En línea]. Disponible: `https://nvd.nist.gov/`.

[7] Cybersecurity and Infrastructure Security Agency, "Known Exploited Vulnerabilities Catalog." [En línea]. Disponible: `https://www.cisa.gov/known-exploited-vulnerabilities-catalog`.

[8] FIRST, "Exploit Prediction Scoring System." [En línea]. Disponible: `https://www.first.org/epss/`.

[9] Joint Task Force, *Risk Management Framework for Information Systems and Organizations: A System Life Cycle Approach for Security and Privacy*, NIST SP 800-37, rev. 2, 2018, doi: 10.6028/NIST.SP.800-37r2.

[10] G. Cudco, *Desarrollo de Software Seguro. Unidad 1: Introducción y estudio de vulnerabilidades*, Tema 1, material de clase, Universidad de las Fuerzas Armadas – ESPE, 2026.

[11] S. Rose, O. Borchert, S. Mitchell y S. Connelly, *Zero Trust Architecture*, NIST SP 800-207, 2020, doi: 10.6028/NIST.SP.800-207.

[12] OWASP Foundation, "OWASP Top 10:2025," 2025. [En línea]. Disponible: `https://top10.owasp.org/2025/`.

[13] OWASP Foundation, "OWASP API Security Top 10 – 2023." [En línea]. Disponible: `https://owasp.org/API-Security/`.

[14] G. Cudco, *Desarrollo de Software Seguro. Unidad 1: Introducción y estudio de vulnerabilidades*, Tema 2: Amenazas a la seguridad del software, material de clase, Universidad de las Fuerzas Armadas – ESPE, 2026.

[15] OWASP Foundation, "Cross Site Scripting Prevention Cheat Sheet." [En línea]. Disponible: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.

[16] OWASP Foundation, "DOM based XSS Prevention Cheat Sheet." [En línea]. Disponible: https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html.

[17] Microsoft, "Threat Modeling Tool," Microsoft Learn. [En línea]. Disponible: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool.

[18] A. Shostack, *Threat Modeling: Designing for Security*. John Wiley & Sons, 2014.

[19] National Institute of Standards and Technology, *The NIST Cybersecurity Framework (CSF) 2.0*, NIST CSWP 29, 2024, doi: 10.6028/NIST.CSWP.29.

[20] ISO/IEC, *ISO/IEC 27001:2022: Information security, cybersecurity and privacy protection — Information security management systems — Requirements*, 2022. [En línea]. Disponible: https://www.iso.org/es/norma/27001.

[21] ISO/IEC, *ISO/IEC 27002:2022: Information security, cybersecurity and privacy protection — Information security controls*, 2022. [En línea]. Disponible: https://www.iso.org/standard/75652.html.

[22] ISO/IEC, *ISO/IEC 27005:2022: Information security, cybersecurity and privacy protection — Guidance on managing information security risks*, 2022. [En línea]. Disponible: https://www.iso.org/standard/80585.html.